Pontifícia Universidade Católica de Minas Gerais
Curso de Engenharia de Software
Disciplina: Projeto de Software
Profa. Milena Menezes Adão

Laboratório 3 – Sprint 2

Sistema de Moeda Estudantil

Lucas Lima Magalhães Lafetá Chaves

Outubro de 2026

github.com/lucaslmchaves/sistema-moeda-estudantil (branch sprint-2)

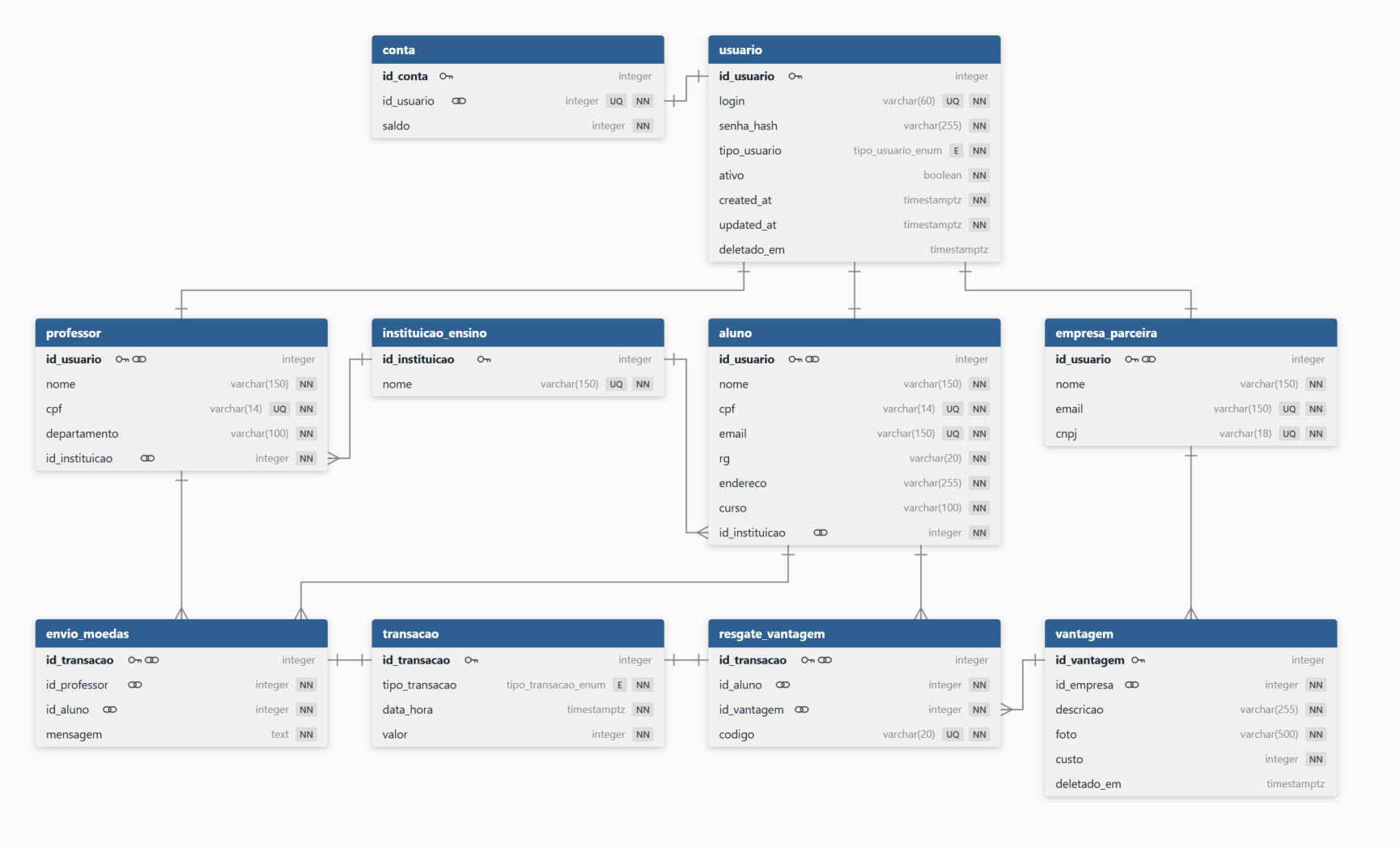
Este documento reúne a entrega da Sprint 2 do Sistema de Moeda Estudantil: o modelo ER, a estratégia de acesso ao banco de dados e a versão inicial dos CRUDs de aluno e de empresa parceira, com o front-end conversando com o back-end. A versão final dos CRUDs e a apresentação da arquitetura ficam para a Sprint 3.

1. Modelo ER

O banco é relacional (PostgreSQL). Escolhi esse tipo porque o saldo e as transações precisam ser gravados juntos: quando um professor envia moedas, o débito dele, o crédito do aluno e o registro da transação acontecem todos ou nenhum. O modelo está na próxima página, e estas foram as principais decisões:

- A herança do diagrama de classes virou tabelas ligadas pela chave. `aluno`, `professor` e `empresa_parceira` têm como chave primária o `id_usuario`, que também é chave estrangeira para `usuario`. O mesmo vale para `envio_moedas` e `resgate_vantagem` em relação a `transacao`.
- `MembroInstituicao` é abstrata e não tem tabela: nome, CPF e instituição aparecem em `aluno` e em `professor`.
- Cada aluno e cada professor tem uma conta (`conta`, com `id_usuario` único), e o saldo não pode ficar negativo.
- Os tipos de usuário e de transação são enums do próprio banco.
- Excluir um cadastro é uma exclusão lógica (coluna `deletado_em`), porque apagar de verdade destruiria o histórico de transações.
- CPF, CNPJ, e-mail e login são únicos. CPF e CNPJ são guardados só com os dígitos.
- A vantagem guarda a foto como um caminho, e o arquivo fica fora do banco. O valor de cada troca será gravado na transação, para o histórico não mudar se a vantagem ficar mais cara depois.

Modelo ER


conta
id_conta integer
id_usuario integer UQ NN
saldo integer NN
usuario
id_usuario integer
login varchar(60) UQ NN
senha_hash varchar(255) NN
tipo_usuario tipo_usuario_enum E NN
ativo boolean NN
created_at timestamptz NN
updated_at timestamptz NN
deletado_em timestamptz
professor
id_usuario integer
nome varchar(150) NN
cpf varchar(14) UQ NN
departamento varchar(100) NN
id_instituicao integer NN
instituicao_ensino
id_instituicao integer
nome varchar(150) UQ NN
aluno
id_usuario integer
nome varchar(150) NN
cpf varchar(14) UQ NN
email varchar(150) UQ NN
rg varchar(20) NN
endereco varchar(255) NN
curso varchar(100) NN
id_instituicao integer NN
empresa_parceira
id_usuario integer
nome varchar(150) NN
email varchar(150) UQ NN
cnpj varchar(18) UQ NN
envio_moedas
id_transacao integer
id_professor integer NN
id_aluno integer NN
mensagem text NN
transacao
id_transacao integer
tipo_transacao tipo_transacao_enum E NN
data_hora timestamptz NN
valor integer NN
resgate_vantagem
id_transacao integer
id_aluno integer NN
id_vantagem integer NN
codigo varchar(20) UQ NN
vantagem
id_vantagem integer
id_empresa integer NN
descricao varchar(255) NN
foto varchar(500) NN
custo integer NN
deletado_em timestamptz

2. Estratégia de acesso ao banco de dados

Usei o padrão Repository em cima de um ORM (SQLAlchemy). O caminho de uma requisição é Controller, Serviço, Repositório, ORM e Banco:

- o **controller** recebe a requisição, confere o formato dos dados e devolve a resposta;
- o **serviço** aplica as regras de negócio: CPF e CNPJ válidos, dados únicos, senha guardada com bcrypt, conta criada junto com o aluno;
- o **repositório** só consulta e grava, sem decidir nada;
- o **ORM** liga as classes às tabelas e gera o SQL.

Escolhi Repository com ORM, e não DAO com SQL escrito à mão, porque o ORM já resolve o mapeamento da herança em tabelas ligadas e evita repetir SQL. Mesmo assim, o repositório esconde o banco do resto do sistema, então trocar de banco mexeria só nessa camada.

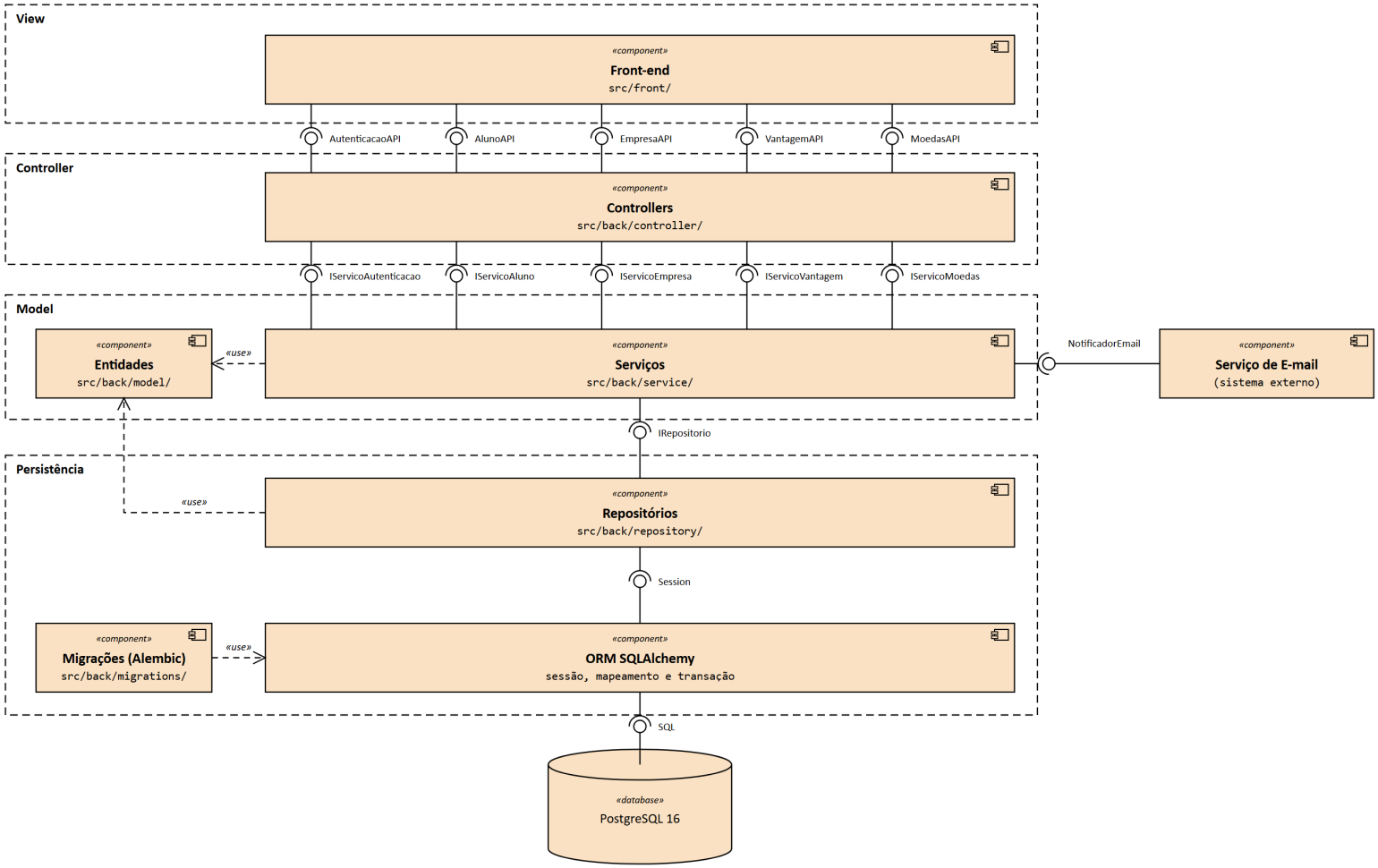
Cada requisição usa uma sessão do banco: se tudo dá certo, é feito o commit, e se algo falha, o rollback. Por isso cadastrar um aluno cria o usuário, o aluno e a conta na mesma transação. As tabelas são criadas por migrações do Alembic, que ficam versionadas no repositório, e as instituições e os professores pré-cadastrados entram por um script de carga inicial que pode rodar mais de uma vez sem duplicar nada.

Os serviços não conhecem o repositório concreto: eles dependem de interfaces (`IRepositorioAluno`, `IRepositorioEmpresa` e `IRepositorioInstituicao`), que os repositórios do ORM realizam. Com isso, os testes dos serviços rodam com repositórios falsos em memória, sem banco, e trocar a forma de acesso aos dados não muda as regras de negócio.

O back-end tem 111 testes automatizados (modelos, repositórios, serviços e API) e o front-end tem 73 (funções, hooks e componentes).

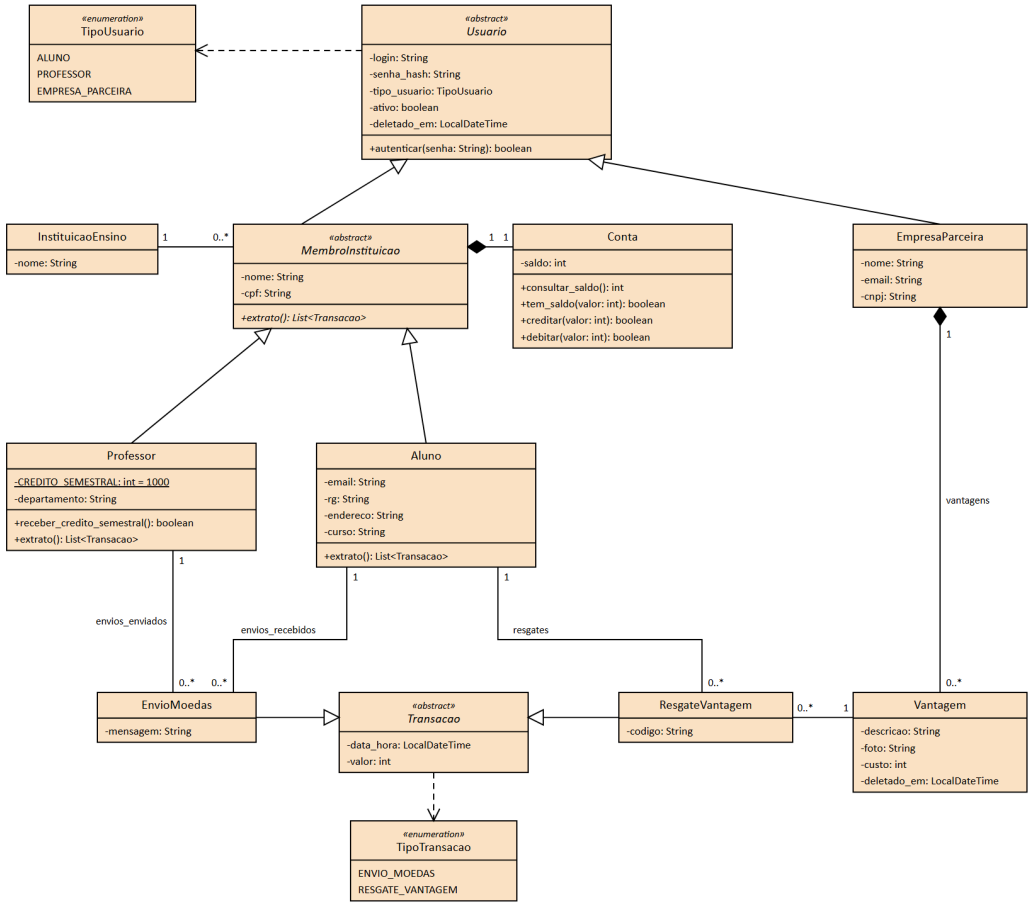
3. Diagrama de Componentes

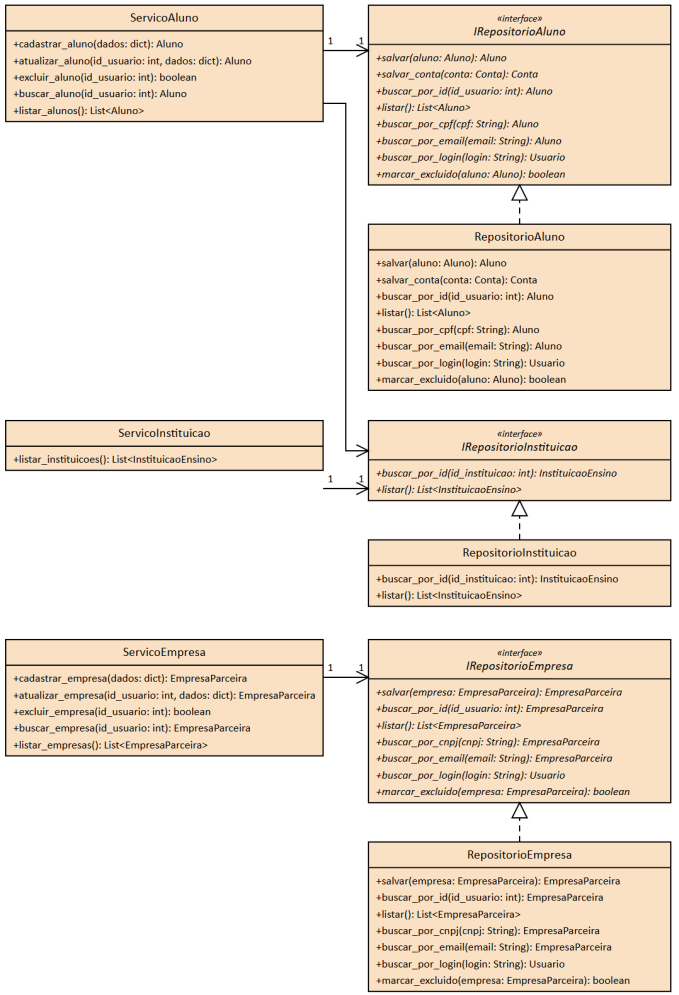
O diagrama mantém a arquitetura MVC da Sprint 1 e detalha a persistência: os repositórios usam o ORM (a sessão), as migrações do Alembic também usam o ORM, e o ORM fala com o PostgreSQL. Cada componente é uma pasta do projeto.



4. Diagrama de Classes

Este diagrama mostra só o que está implementado nesta sprint, com os mesmos nomes do código. Os serviços de autenticação, de vantagem e de moedas e a notificação por e-mail entram na Sprint 3. Os repositórios ocupam o lugar das listas estáticas do diagrama da Sprint 1: agora quem guarda e entrega as coleções é o banco, por meio deles, e cada serviço depende só da interface do seu repositório. As listas de envios, resgates e vantagens aparecem como papéis das associações, e o `extrato()` de aluno e de professor usa essas listas.





5. CRUDs de aluno e empresa parceira

O front-end é feito em React com Tailwind CSS, e a lógica de cada tela fica em hooks, separada dos componentes. Ele conversa com a API por HTTP. As rotas são as mesmas para aluno e para empresa parceira (`/alunos` e `/empresas`):

Rota	Ação	Resposta
`POST /alunos`	Cadastrar	201
`GET /alunos` e `GET /alunos/{id}`	Listar e consultar	200 (ou 404)
`PUT /alunos/{id}`	Atualizar (o login não muda)	200
`DELETE /alunos/{id}`	Excluir (exclusão lógica)	204
`GET /instituicoes`	Listar as instituições do cadastro	200

Os erros de validação vêm do servidor e aparecem na tela: dado inválido (422), dado repetido (409) e cadastro não encontrado (404). O CPF e o CNPJ podem ser digitados com ou sem pontuação; o servidor guarda só os dígitos e a tela mostra com máscara.

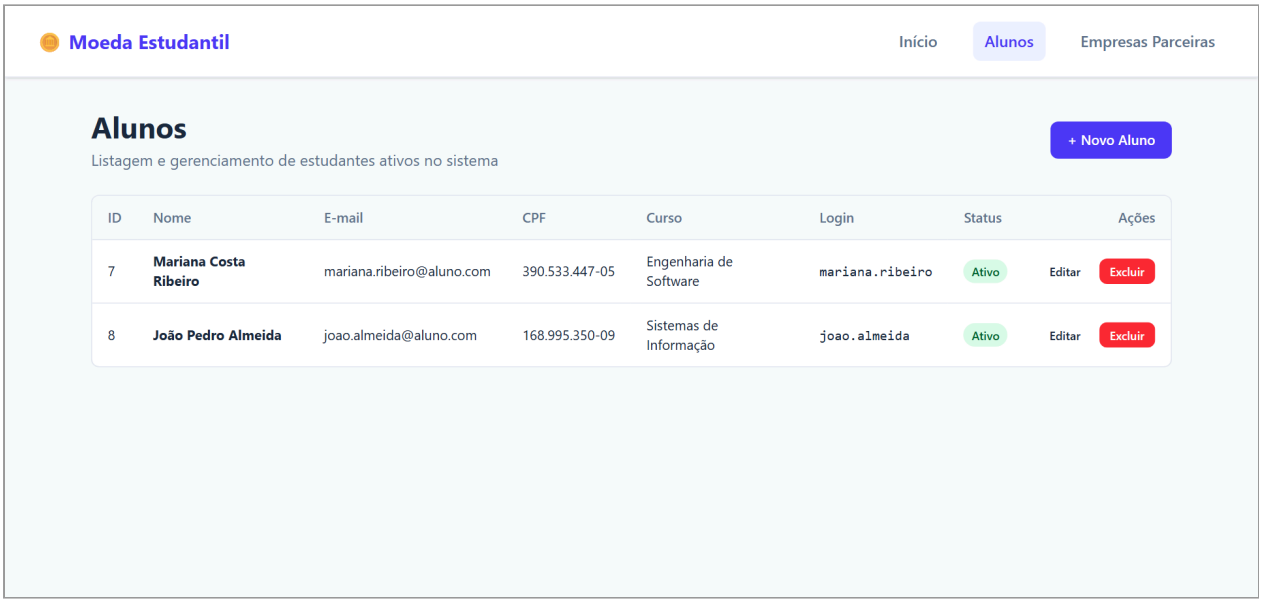


Figura 1 – Lista de alunos

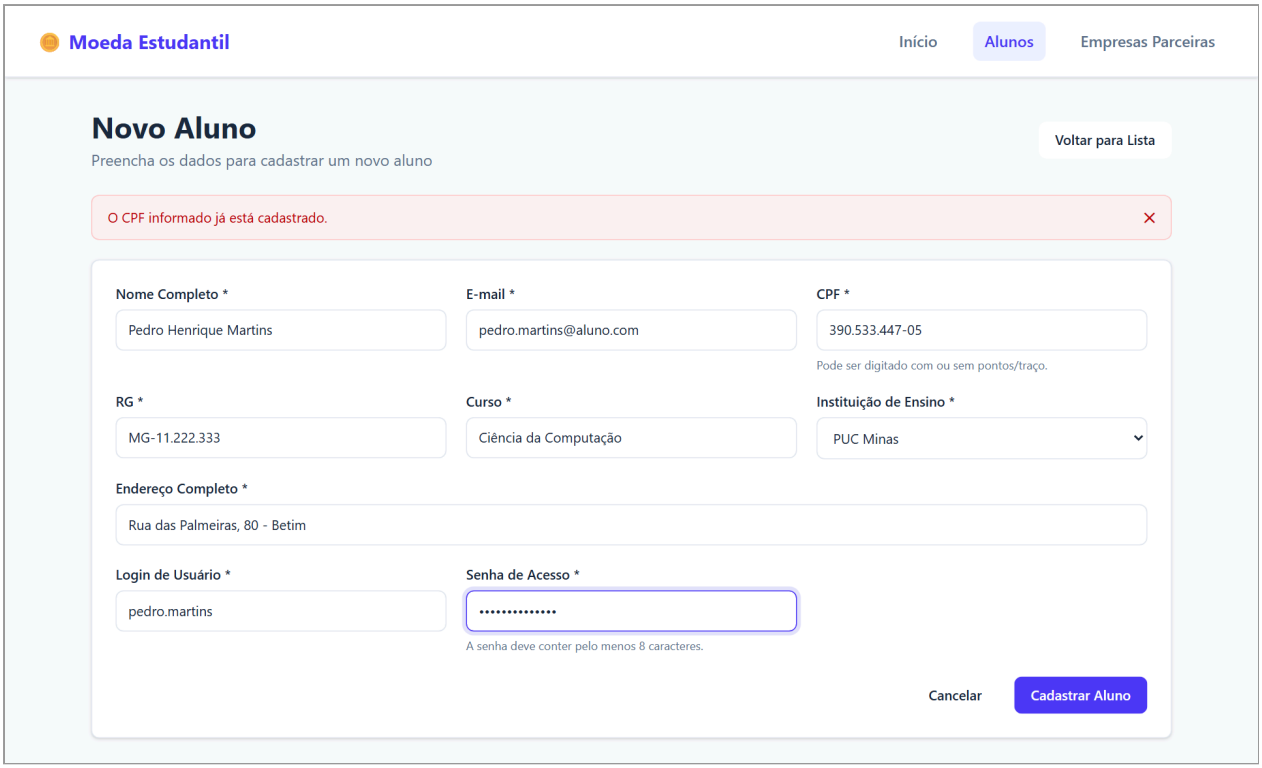


Figura 2 – Cadastro de aluno com CPF já cadastrado: o erro 409 da API aparece na tela

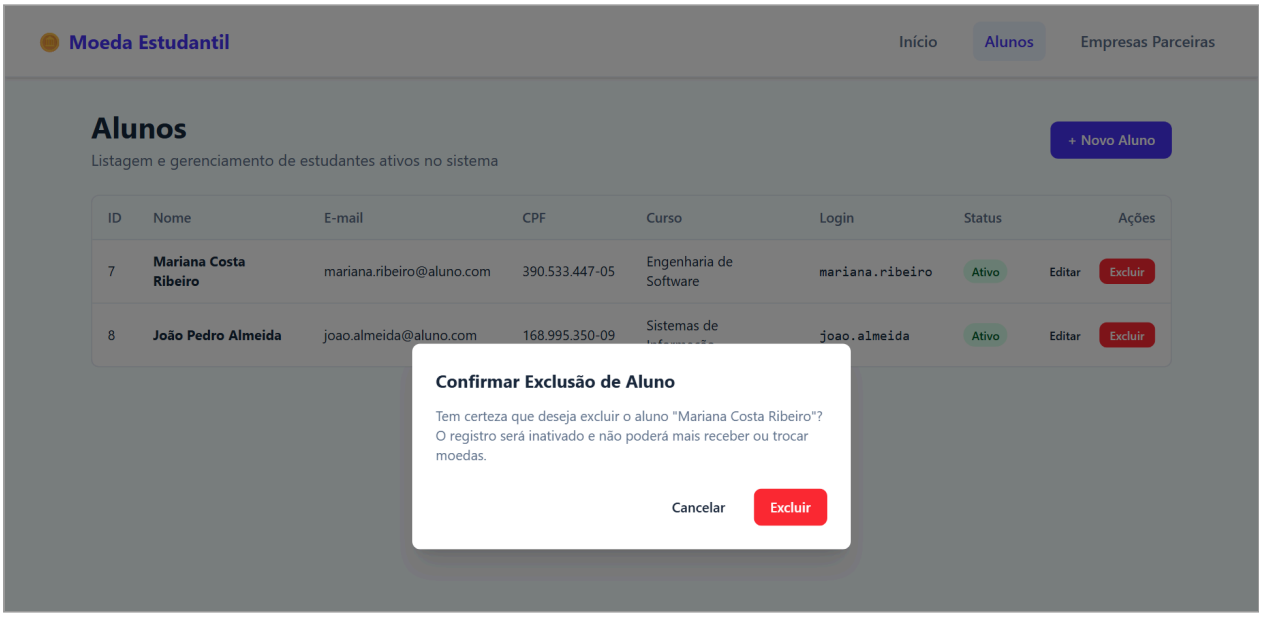


Figura 3 – Confirmação antes de excluir um aluno

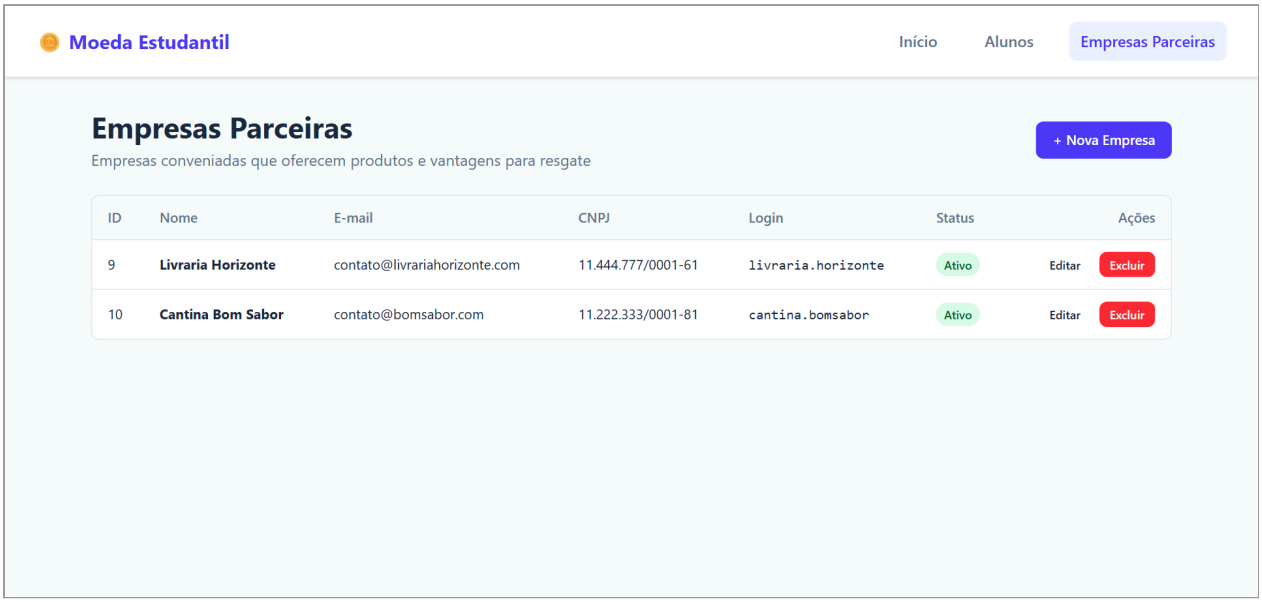


Figura 4 – Lista de empresas parceiras

6. Como executar

Com o Docker instalado, na pasta do projeto:

```
docker compose up -d --build
```

O front-end abre em `http://localhost:5173` e a documentação da API em `http://localhost:8000/docs`. As migrações e os dados iniciais (2 instituições e 3 professores) são aplicados ao subir a API. Os testes rodam com `docker compose exec api pytest -q`, e o script `scripts/verificar.sh` faz um CRUD completo de aluno e de empresa para conferir a stack inteira.